



Princess Nourah bint
Abdulrahman University

IS350 Project Management

CloudGuard AI Report

Group ID: 2

Section: 6w3

No.	Student Name	Student Number
1	هند بن سنيد	445008343
2	شذى طلال باخشوين	445008415
3	روان سعيد السلمي	445008450
4	شجون علي الشهري	445008413
5	رزان القحطاني	445008485
6	امجاد الخريجي	445008473

Contents

Introduction:	2
Problem Statement:	2
Methodology with Justification:	3
Project Charter:	4
Project Requirements:	5
Project Scope Statement:	7
WBS and WBS dictionary:	9
RACI Chart:	15
Gantt Chart:	18
Risk Register:	19
References:	0

Table of figures

Figure 1: Project Charter for CloudGuardAI	4
Figure 2: WBS for CloudGuard AI	10

Table:

Table 1: WBS Dictionary	14
Table 2: RACI Chart for CloudGuard AI Project	17
Table 3: Risk Register	19

Introduction:

Cloud computing is widely used by organizations to store data and run applications, which makes cloud security an important concern. Security services such as Amazon GuardDuty, which is a threat detection service provided by Amazon Web Services (AWS), help monitor cloud environments and detect potential threats. However, these systems often generate a large number of alerts, many of which are false positives, making it difficult for security teams to quickly identify real threats.

This project introduces CloudGuard AI: Intelligent Cloud Threat Detection, an improved concept designed to enhance cloud threat monitoring by utilizing Artificial Intelligence to assist security teams in analyzing alerts and improving the efficiency of threat detection within cloud environments. [1]

Problem Statement:

With the rapid growth of cloud computing, many organizations rely on cloud security services such as Amazon GuardDuty to monitor system activities and detect potential security threats. However, these systems often generate a large number of alerts, many of which are false positives, making it difficult for cybersecurity teams to quickly identify real threats. Security analysts are required to manually review many alerts, which consume time and resources and may delay responses to actual security incidents.

Additionally, existing systems do not always provide clear classification or analysis of alerts, which makes it harder for security teams to understand patterns of suspicious activities. Therefore, there is a need to improve the current system by introducing several enhancements.

First, Artificial Intelligence (AI) can be used to automatically classify alerts into categories such as normal activity, suspicious behavior, or potential attacks, helping analysts focus on critical threats. Second, the system can generate monthly summaries of alerts to identify patterns and provide statistics on each alert category. Third, every new IP address detected in the system can be compared with a previous database to determine whether it is known or unknown, allowing security teams to monitor potentially dangerous or unfamiliar activities more effectively. These improvements will help increase the efficiency of threat detection and support faster and more accurate security decision-making.

Methodology with Justification:

The Incremental Model is used in this project because the system consists of several independent features that can be developed and tested step by step. In incremental development, the system is built through a series of smaller components called increments, where each increment adds new functionality to the system until the complete product is achieved. This approach allows teams to implement and test each feature separately before integrating it into the overall system, which helps detect errors early and improve system reliability.

The system will be developed through three main increments: The first increment focuses on implementing the AI-based alert classification feature, which analyzes AWS GuardDuty alerts and categorizes them into Normal, Suspicious, or Attack. The second increment introduces the dashboard and monthly alert summary feature, allowing the security team to view alert priorities and identify patterns in system activity. The third increment implements the IP verification feature, where new IP addresses are compared with a stored database and classified as known or unknown. Each increment will be developed and tested before moving to the next stage to ensure system reliability and proper functionality. [2]

Project Charter:

Project Title : CloudGuard AI : Intelligent Cloud Threat Detection

Date of Authorization: March 5, 2026

Project start Date : March 5, 2026

Project finish date : May 15, 2026

Key Schedule Milestones:

- Implement AI-based alert, March 18, 2026
- Deploy monthly alert summary dashboard, April 12, 2026
- Integrate IP verification system and complete testing and improvements, May 10, 2026
- Final system delivery May 15, 2026

Budget Information: The estimated budget for The CloudGuard AI: Intelligent cloud threat detection is approximately \$9,000, AI-based alert classification is estimated at \$4,000, the suspicious IP address monitoring at \$3,000, and the alert summary dashboard at \$2,000. Most of the costs are related to development and testing efforts required to implement these improvements to Amazon GuardDuty.

Project Manager: Razan Alqahtani, (966) 505989200, 445008485@pnu.edu.sa

Project Objectives: The goal of CloudGuard AI is to improve cloud threat detection by reducing false positive alerts and helping security teams prioritize threats. The system uses AI to classify alerts into Normal, Suspicious, and Attack, provides monthly alert summaries, and verifies new IP addresses against a database to support faster and more effective security monitoring.

Main Project Success Criteria: The project will be considered successful if the system accurately classifies AWS GuardDuty alerts into Normal, Suspicious, and Attack using AI and displays them clearly in a prioritized dashboard. The system should also provide monthly summaries and identify unknown IP addresses to help reduce false positive alerts and improve security monitoring efficiency.

Approach:

- Develop a project plan that defines tasks, milestones, and team responsibilities.
- Analyze AWS GuardDuty alert data and develop an AI model to classify alerts into Normal, Suspicious, or Attack.
- Design and implement a dashboard that displays prioritized alerts, monthly summaries, and IP verification results.
- Conduct testing to ensure accurate alert classification, proper IP monitoring, and reliable system performance.

ROLES AND RESPONSIBILITIES

Name	Role	Position	Contact Information
Hind	System Developer	Backend Developer	445008343@pnu.edu.sa
Amjad	Data Analyst	Security Analyst	445008473@pnu.edu.sa
Razan	Project Manager	Manager	445008485@pnu.edu.sa
Rawan	Dashboard Developer	Frontend Developer	445008450@pnu.edu.sa
Shatha	AI Developer	AI Engineer	445008415@pnu.edu.sa
Shujon	Testing & Documentation	QA Engineer	445008413@pnu.edu.sa

Sign-off: *Hind. Shatha. Rawan. Razan. Shujon. Amjad.*

Comments: "I expect the team to focus on reducing false positive alerts and delivering a clear and reliable dashboard for security teams." -Sara Alqahtani

"The proposed system shows strong potential in supporting security analysts by prioritizing alerts and providing clear summaries of system activity." -Noura Alghamdi

Figure 1: Project Charter for CloudGuardAI

Project Requirements:

Stakeholders and Requirement Elicitation Techniques

1. Stakeholder: Security Analysts

Technique: Interviews

Reason: Security analysts are the primary users of the system and directly interact with alerts. Interviews provide detailed insights into their challenges, such as handling false positives and analyzing large volumes of alerts, which helps define accurate system requirements.

2. Stakeholder: Security Team (System Users)

Technique: Questionnaires

Reason: Since the user group is large, questionnaires are effective for collecting requirements efficiently. This helps identify common needs, preferences, and expectations.

3. Stakeholder: Cloud Security Experts / AWS Specialists

Technique: Benchmarking

Reason: Benchmarking helps compare existing cloud security solutions and AI-based systems. This ensures the proposed system follows best practices and meets industry standards.

4. Stakeholder: IT Administrator

Technique: Observation

Reason: Helps understand how current systems are managed, monitored, and how alerts are handled in real environments.

5. Stakeholder: Project Manager

Technique: Interviews

Reason: The project manager defines project goals, scope, and constraints. Interviews help ensure alignment with project objectives and timeline.

6. Stakeholder: Organization / Client

Technique: Document Analysis

Reason: Helps identify business requirements and security policies.

Software Requirements

- Python (for AI model development)
- AWS Cloud Services (e.g., AWS GuardDuty)
- Dashboard development tools (e.g., React or Power BI)
- Database management system (e.g., MySQL or MongoDB)

Hardware Requirements

- Cloud server or hosting environment
- Developer workstations (computers/laptops)
- Stable internet connection

Functional Requirements

1. The system shall classify alerts using AI into the following categories:
 - Normal
 - Suspicious
 - Attack
2. The system shall provide a dashboard that displays:
 - Alert priorities
 - Monthly summaries
 - Statistical analysis of alerts
3. The system shall verify IP addresses by:
 - Comparing incoming IPs with a stored database
 - Identifying IPs as known or unknown
4. The system shall generate monthly reports to analyze alert patterns and trends.
5. The system shall allow users to view and filter alerts based on priority levels.

Non-Functional Requirements

1. Performance:
The system should process and classify alerts within 2–3 seconds.
2. Accuracy:
The AI model should achieve at least 85% classification accuracy.
3. Security:
The system must ensure secure access using authentication and protect data using encryption techniques.
4. Usability:
The dashboard should provide a clear and user-friendly interface that allows users to easily understand alert priorities.
5. Reliability:
The system should operate with at least 95% uptime and minimal system failures.

Project Scope Statement:

Project Title: CloudGuard AI: Intelligent Cloud Threat Detection **Date Prepared:** 26/4/2026

Project Scope Description:

The CloudGuard AI project aims to enhance cloud security monitoring by improving the functionality of AWS GuardDuty. The system will use Artificial Intelligence to automatically classify security alerts into three categories: Normal, Suspicious, and Attack. It will also provide a user-friendly dashboard that displays alert priorities, monthly summaries, and statistical insights. In addition, the system will include an IP verification feature that compares new IP addresses with a stored database to identify unknown or potentially risky activities. This project focuses on reducing false positive alerts and helping security teams make faster and more accurate decisions.

Project Deliverables:

- AI-based alert classification model
- Interactive dashboard for alert monitoring
- Monthly alert summary reports
- IP verification and monitoring feature
- Database for storing alerts and IP records
- Fully tested and functional system
- Project documentation and final report

Product Acceptance Criteria:

- The system correctly classifies alerts into Normal, Suspicious, and Attack
- The dashboard clearly displays alert priorities and summaries
- The system generates accurate monthly reports
- The IP verification feature correctly identifies known and unknown IPs
- The system meets performance and reliability requirements
- The project is completed within the defined schedule and approved by stakeholders

Project Exclusions:

- Integration with external security systems beyond AWS GuardDuty
- Real-time deployment in a production cloud environment
- Advanced AI model optimization beyond basic classification
- Continuous system maintenance after project completion

Project Constraints:

- Limited project timeline (March 2026 – May 2026)
- Limited budget and resources
- Dependence on available AWS GuardDuty data
- Development limited to academic project scope

Project Assumptions:

- AWS GuardDuty data is available for analysis and testing
- Team members have basic knowledge of AI and cloud systems
- Required tools and technologies are accessible
- Stakeholders will provide feedback when needed

WBS and WBS dictionary:

WBS: The CloudGuard AI project uses a Work Breakdown Structure (WBS) to organize core deliverables like AI alert classification and IP verification into a logical sequence. This structure ensures all components are clearly defined, streamlining planning and management throughout the project lifecycle.

- **Tabular**

- 1.1. Project Initiation (2 days)

- 1.1.1. Define project scope and objectives

- 1.1.2. Identify stakeholders and their roles

- 1.1.3. Develop project charter

- 1.2. Project Planning (4 days)

- 1.2.1. Define system requirements (functional and non-functional)

- 1.2.2. Identify system components (AI, Dashboard, IP Verification)

- 1.2.3. Develop project plan and timeline

- 1.2.4. Define system features and functionality

- 1.3. System Design (3 days)

- 1.3.1. Design system architecture

- 1.3.2. Design database structure (alerts and IPs)

- 1.3.3. Define system workflows and interactions

- 1.4. System Development and integration (5 weeks)

- 1.4.1. AI Alert Classification System

- 1.4.1.1. Collect and preprocess alert data

- 1.4.1.2. Develop and train AI model

- 1.4.1.3. Evaluate model accuracy

- 1.4.1.4. Integrate AI model into the system

- 1.4.2. Database Development

- 1.4.2.1. Create alert database

- 1.4.2.2. Create IP database

- 1.4.2.3. Implement data storage and retrieval

- 1.4.3. Dashboard and Reporting System

- 1.4.3.1. Design dashboard interface

- 1.4.3.2. Implement alert prioritization

- 1.4.3.3. Develop visualization features

- 1.4.3.4. Generate monthly reports

- 1.4.4. IP Verification System

- 1.4.4.1. Store and manage known Ips

- 1.4.4.2. Compare incoming IPs with database

- 1.4.4.3. Identify known and unknown Ips

- 1.4.4.4. Integrate results with dashboard

- 1.5. System Testing and Validation (3 weeks)
 - 1.5.1. Test AI classification system
 - 1.5.2. Test dashboard functionality
 - 1.5.3. Test IP verification system
 - 1.5.4. Perform integration and system validation testing
- 1.6. System Deployment and Documentation (5 days)
 - 1.6.1. Deploy system
 - 1.6.2. Prepare technical documentation
 - 1.6.3. Finalize project deliverables

- **Graph**

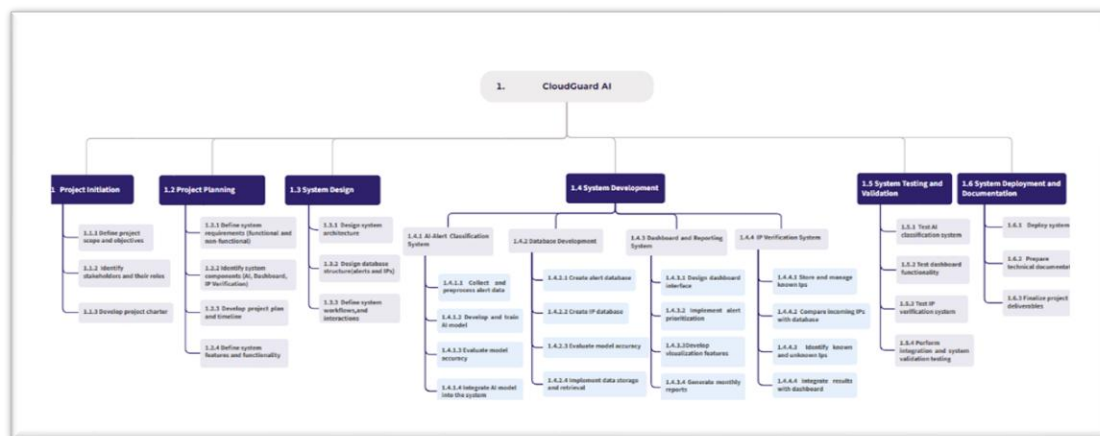


Figure 2: WBS for CloudGuard AI. [Click here.](#)

WBS dictionary:

Code	Name	Description
1.1	Project Initiation	This phase focuses on formally starting the project by defining its objectives, identifying stakeholders, and obtaining approval to proceed. It ensures that the project has a clear direction and official authorization.
1.1.1	Define project scope and objectives	Define the project goals, scope boundaries, expected outcomes, and overall objectives.
1.1.2	Identify stakeholders and their roles	Identify all project stakeholders and define their responsibilities and involvement.
1.1.3	Develop project charter	Prepare the official project charter including project overview, objectives, and approvals.
1.2	Project Planning	This phase defines how the project will be executed by identifying requirements, components, and developing a clear project plan with tasks and timelines.
1.2.1	Define system requirements	Gather and document both functional and non-functional system requirements.
1.2.2	Identify system components	Identify the major system components such as AI classification, dashboard, and IP verification modules.
1.2.3	Develop project plan and timeline	Create a detailed plan outlining tasks, schedule, and milestones to guide project execution.
1.2.4	Define system features and functionality	Specify the main system features and expected functionalities.
1.3	System Design	This phase focuses on designing the technical structure of the system, including architecture, database, and workflows to ensure efficient implementation.
1.3.1	Design system architecture	Design the overall system architecture including servers, databases, and integrations.
1.3.2	Design database structure	Design the database schema for storing alerts and IP address information.

1.3.3	Define system workflows and interactions	Define workflows and interactions between users and system components.
1.4	System Development	This phase involves building and implementing all system components based on the defined design and requirements.
1.4.1	AI Alert Classification System	Develop an AI-based system for classifying and prioritizing security alerts.
1.4.1.1	Collect and preprocess alert data	Gather alert data and clean it to make it suitable for training the AI model.
1.4.1.2	Develop and train AI model	Build and train a machine learning model to classify alerts accurately.
1.4.1.3	Evaluate model accuracy	Test the model using evaluation metrics to ensure it meets required accuracy levels.
1.4.1.4	Integrate AI model into the system	Integrate the trained model into the system to automatically classify incoming alerts.
1.4.2	Database Development	Develop databases required for storing alerts and IP address records.
1.4.2.1	Create Alert Database	Create a database structure for storing security alerts.
1.4.2.2	Create IP Database	Create a database for managing trusted and suspicious IP addresses.
1.4.2.3	Implement Data Storage and Retrieval	Implement efficient mechanisms for storing and retrieving data.
1.4.3	Dashboard and Reporting System	Develop a user interface that displays alerts, supports monitoring, and generates reports.
1.4.3.1	Design dashboard interface	Create a clear and user-friendly interface for displaying system data and alerts.

1.4.3.2	Implement alert prioritization	Display alerts based on their severity levels to help users focus on critical issues.
1.4.3.3	Develop visualization features	Create charts and visualization tools for displaying system data.
1.4.3.4	Generate monthly reports	Develop automated monthly report generation features.
1.4.4	IP Verification System	Develop a system that verifies IP addresses by comparing them with stored records.
1.4.4.1	Store and manage known IPs	Maintain a database of trusted or known IP addresses.
1.4.4.2	Compare incoming IPs with database	Compare incoming IP addresses against stored records.
1.4.4.3	Identify known and unknown IPs	Classify IP addresses as known or unknown based on comparison results.
1.4.4.4	Integrate results with dashboard	Display IP verification results within the dashboard interface.
1.5	System Testing and Validation	This phase ensures that the system works correctly, meets requirements, and performs efficiently under different conditions.
1.5.1	Test AI classification system	Verify that the AI model correctly classifies alerts.
1.5.2	Test dashboard functionality	Ensure that all dashboard features work as expected.
1.5.3	Test IP verification system	Validate the accuracy of the IP verification process.
1.5.4	Perform integration and system validation testing	Ensure all system components work together and validate overall system performance and reliability.

1.6	System Deployment and Documentation	This phase includes releasing the system and preparing all required documentation for final delivery.
1.6.1	Deploy system	Deploy the completed CloudGuard system into the production environment.
1.6.2	Prepare technical documentation	Prepare technical documentation, configuration guides, and user manuals.
1.6.3	Finalize project deliverables	Review and submit all final project deliverables and documentation.

Table 1: WBS Dictionary

RACI Chart:

Task Name	Project Manager	Backend Developer	Frontend Developer	AI Engineer	Security Analyst	QA Engineer
Define project scope and objectives	A/R				C	
Identify stakeholders and their roles	A/R					
Develop project charter	A/R				C	
Define system requirements	A				R	
Identify system components	A	R		C	C	
Develop project plan and timeline	A/R					
Define system functionality	A	C	C		R	
Design system architecture	A	R		C	C	
Design database structure		A/R			C	
Define workflows and interactions	A		R			
Collect and preprocess alert data				A/R	C	

Develop and train AI model				A/R	C	
Evaluate model accuracy				A/R		C
Integrate AI model into the system	A	R		R		
Create alert database		A/R				
Create IP database		A/R			C	
Implement data storage and retrieval		A/R				
Design dashboard interface			A/R			
Implement alert prioritization			A/R	C		
Develop visualization features			A/R			
Generate monthly reports			A/R			
Store and manage known IPs		R			A	
Compare incoming IPs with database		R			A	
Identify known and unknown IPs					A/R	

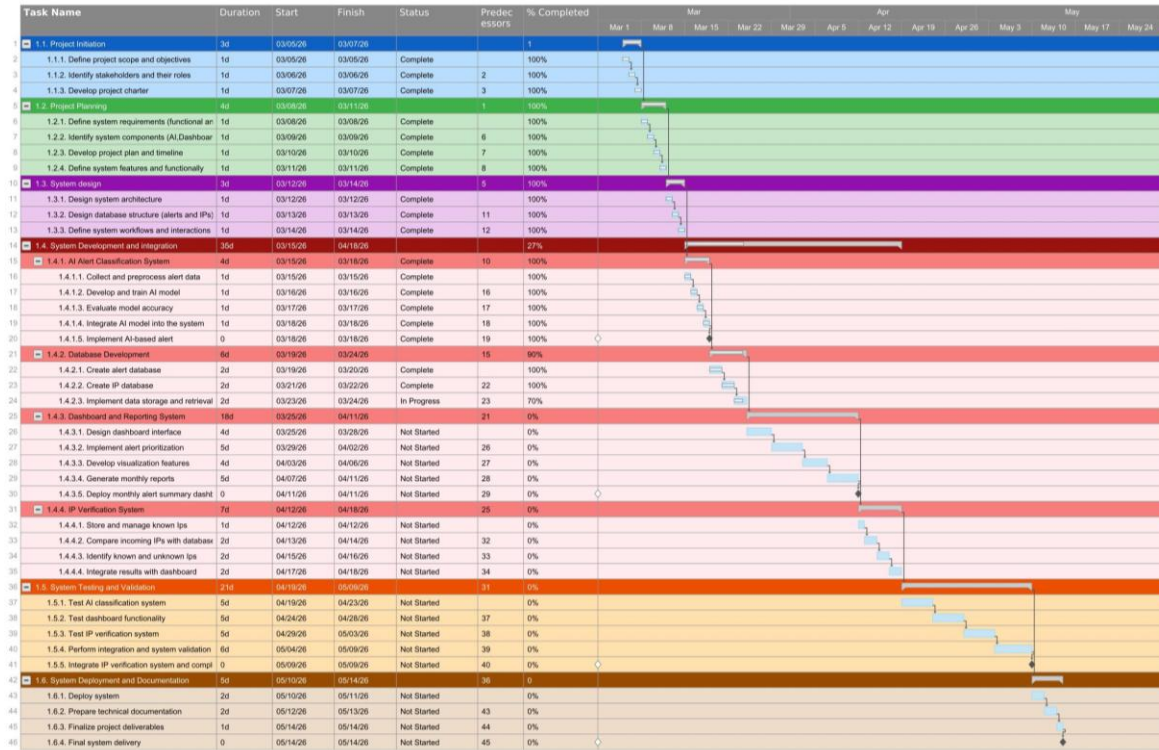
Integrate results with dashboard		R	R		A	
Test AI classification system				C		A/R
Test dashboard functionality			C			A/R
Test IP verification system		C			C	A/R
Perform integration and system validation testing	A					R
Deploy system	A	R	C			
Prepare technical documentation						A/R
Finalize project deliverables	A					R

Table 2: RACI Chart for CloudGuard AI Project

Key assumptions:

- **Project Manager:** Responsible for managing the project schedule, coordinating tasks, tracking milestones, and ensuring timely delivery of the final project.
- **Backend Developer:** Responsible for database development, backend integration, and implementing data storage and retrieval functionalities.
- **Frontend Developer:** Responsible for designing and developing the dashboard interface, visualization features, and reporting components.
- **AI Engineer:** Responsible for collecting and preprocessing alert data, developing and training the AI model, and evaluating model performance.
- **Security Analyst:** Responsible for defining system requirements related to security, managing IP verification processes, and validating suspicious IP detection.
- **QA & Documentation Engineer:** Responsible for system testing, validation activities, technical documentation, and ensuring the quality of final deliverables.

Gantt Chart:



Risk Registre:

NO.	Risk	Description	Positive or Negative risk	Response Strategy
R1	Inaccurate AI Threat Detection	The system may generate false alerts or fail to detect some threats accurately.	Negative	Risk Mitigation: Continuously train and test the AI model using updated threat data.
R2	Project Delays Due to Technical Challenges	Unexpected technical issues may delay development and implementation phases.	Negative	Risk Mitigation: Monitor project progress regularly and maintain a clear project schedule.
R3	Reduced Human Effort in Threat Monitoring	The system can automate threat analysis and reduce manual monitoring tasks.	Positive	Risk Exploitation: Demonstrate how automation improves operational efficiency and saves time.
R4	Increased Interest in AI-Based Security Solutions	Project success may encourage future adoption of similar AI security solutions.	Positive	Risk Exploitation: Use the project as a foundation for future AI cloud security improvements.
R5	Weak Network Connection During Cloud Testing	Network or internet connection issues may affect testing operations or access to cloud services.	Negative	Risk Mitigation: Use a stable internet connection and prepare backup plans during testing.
R6	Improved Threat Detection Speed and Accuracy	AI enhancement can improve the efficiency and accuracy of threat detection.	Positive	Risk Exploitation: Highlight the improved detection capabilities in presentations and reports.

Table 3: Risk Registre



Princess Nourah bint
Abdulrahman University

References:

- [1] Intelligent threat detection – amazon guardduty – AWS,
<https://aws.amazon.com/guardduty/> (accessed Mar. 8, 2026).
- [2] “Incremental build model,” Wikipedia,
https://en.wikipedia.org/wiki/Incremental_build_model (accessed Mar. 8, 2026).